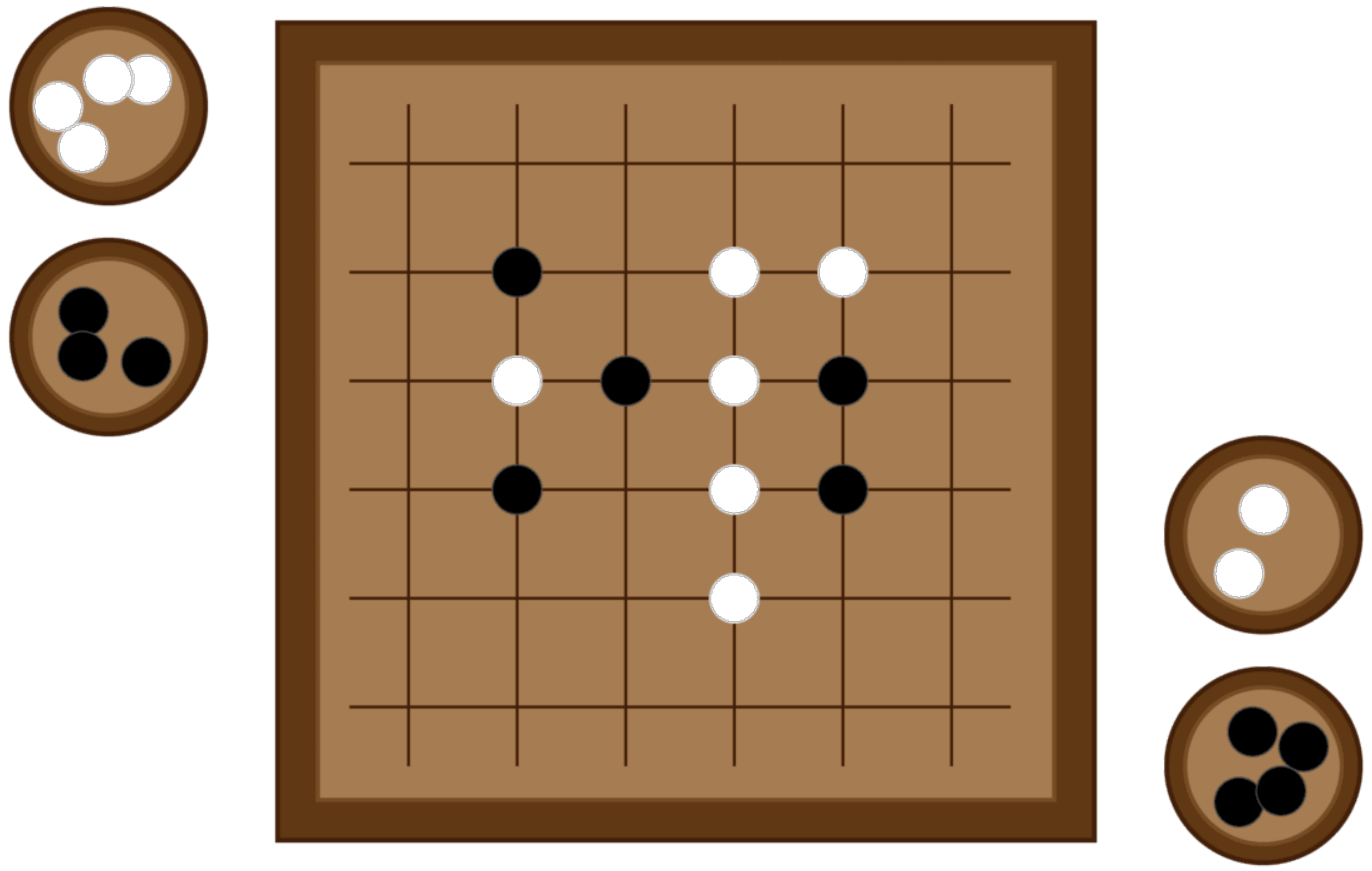


Mastering the game of Go without Human Knowledge

David Silver, Julian Schrittwieser, Karen Simonyan, ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas baker, Matthew Lai, Adrian bolton, Yutian chen, Timothy Lillicrap, Fan Hui, Laurent Sifre, George van den Driessche, Thore Graepel & Demis Hassabis

Presented by: Alessia Crimaldi, Davide Luccioli

AlphaGo Zero: learning from first principles

- Tabula rasa
- Based solely on Reinforcement Learning
- Learns to play without:
  - Human knowledge
  - Human guidance
  - Domain knowledge beyond game rules

Motivation 1

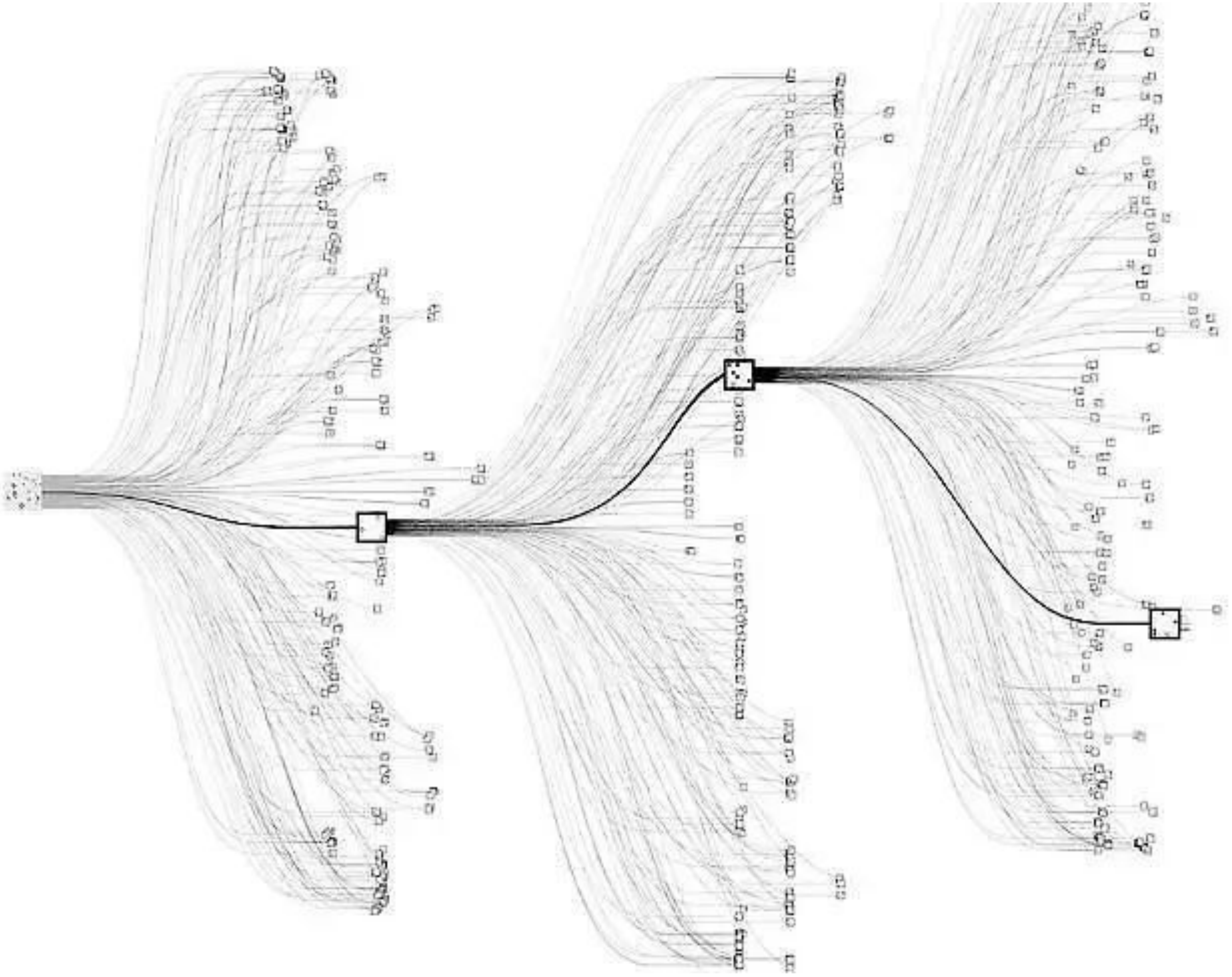
- The reason why AI researchers care about the game of Go is that it is incredibly complex and it is hard for computers

- **LARGE BOARD:**
  the number of possible configurations on the board is more than the number of atoms in the universe

  19×19 **board** $\rightarrow 3^{19\times 19} \approx 10^{170}$

- **LARGE BRANCHING FACTOR:**
  there are about 150-250 moves on average playable from a given game state

- Such a challenging domain in terms of human intellect demands precise and sophisticated lookahead across a vast search space

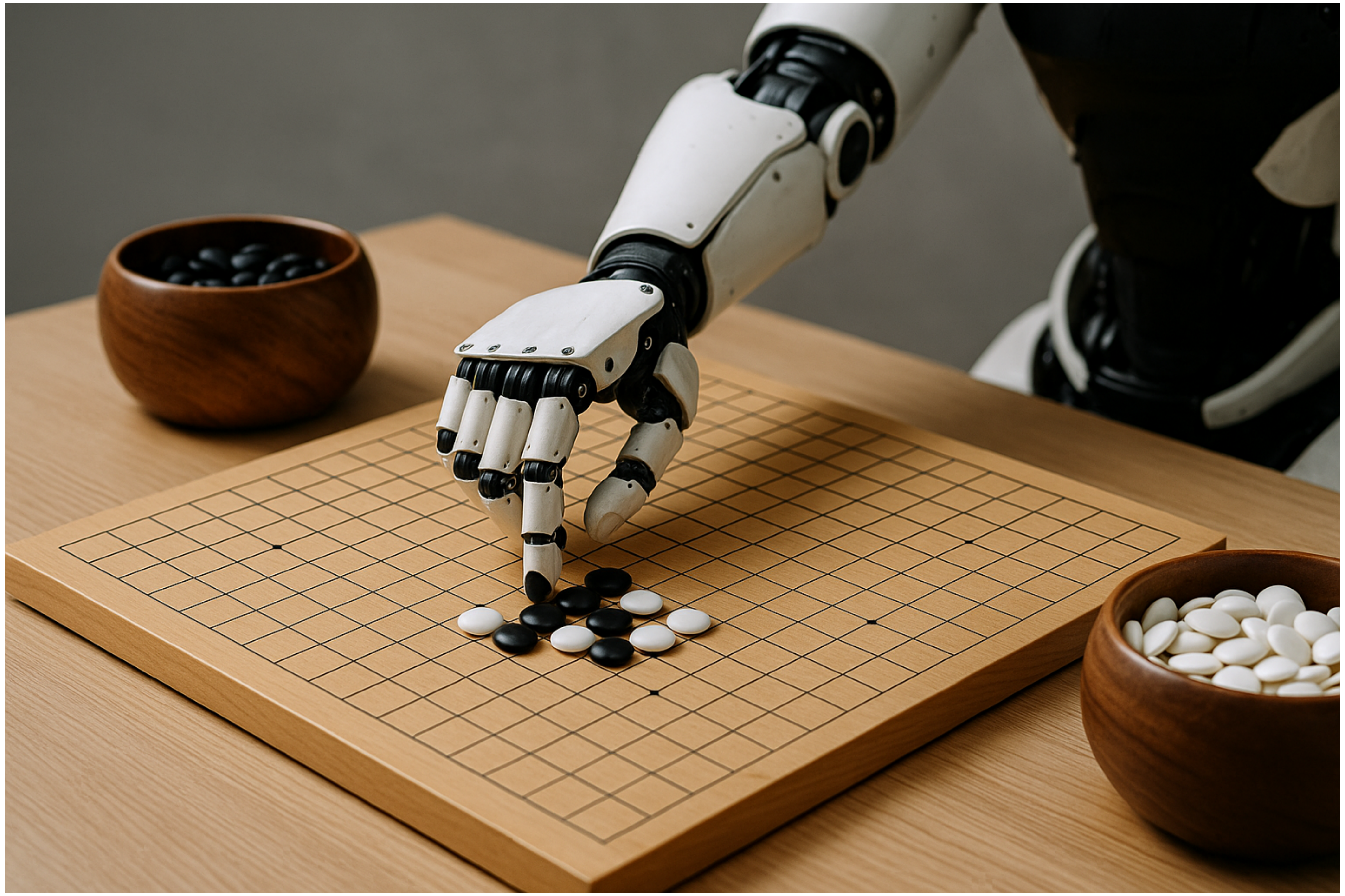


Credit: Nikhil Cheerla, 2018

Motivation 2

- We use neural networks trained by **reinforcement learning** because:
  - Human data for training is often *expensive*, *unreliable*, or simply *unavailable*
  - Even when human data exists, it can limit the performance of the systems trained on it

- Reinforcement learning allows systems to **learn from their own experience**, enabling them to:
  - Surpass human capabilities
  - Perform in domains where human expertise is lacking



Comparison with AlphaGo Fan

AlphaGo Fan

- First algorithm to achieve superhuman performance in Go
- **Training:** Supervised learning + self-play
- **Model:** Policy and value networks
- **Input:** Hand-crafted features
- **Rollouts** are used, lookahead search provided from the trained network combined with Monte Carlo Tree Search (MCTS)

AlphaGo Zero

- Improves upon the original architecture of AlphaGo Fan
- **Training:** Entirely self-play, starting from random play
- **Model:** Single Neural Network for both policy and value
- **Input:** Raw game state representation
- **No rollouts**, lookahead search incorporated in the RL algorithm inside the training loop

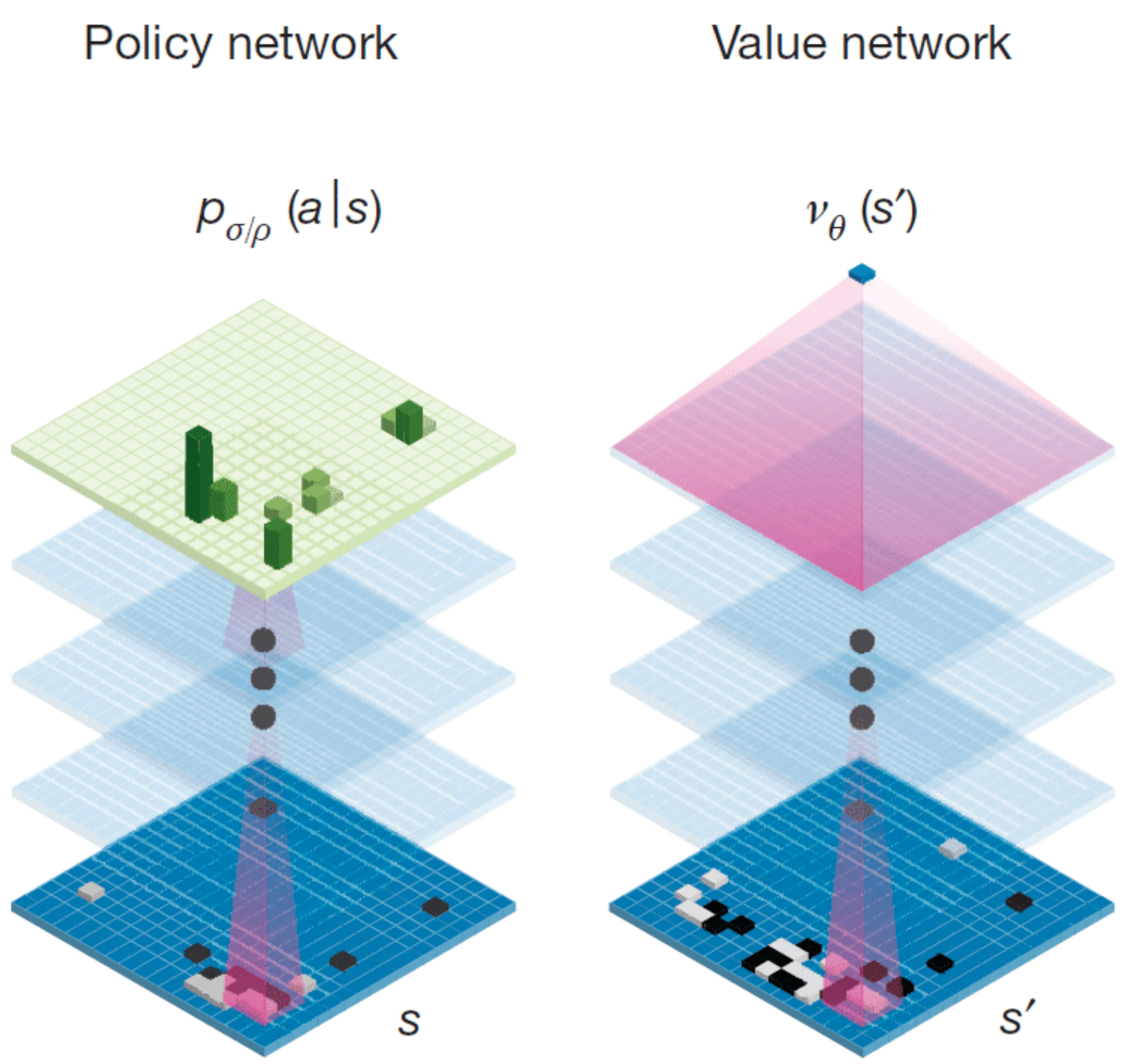
Key characteristics

AlphaGo Zero combines:

- A novel self-play RL algorithm
- Neural network (NN) for function estimation that, given a board's state:
  - Predicts move selection → policy network
  - Predict the game winner → value network
- Monte Carlo Tree Search (MCTS)

Neural Network

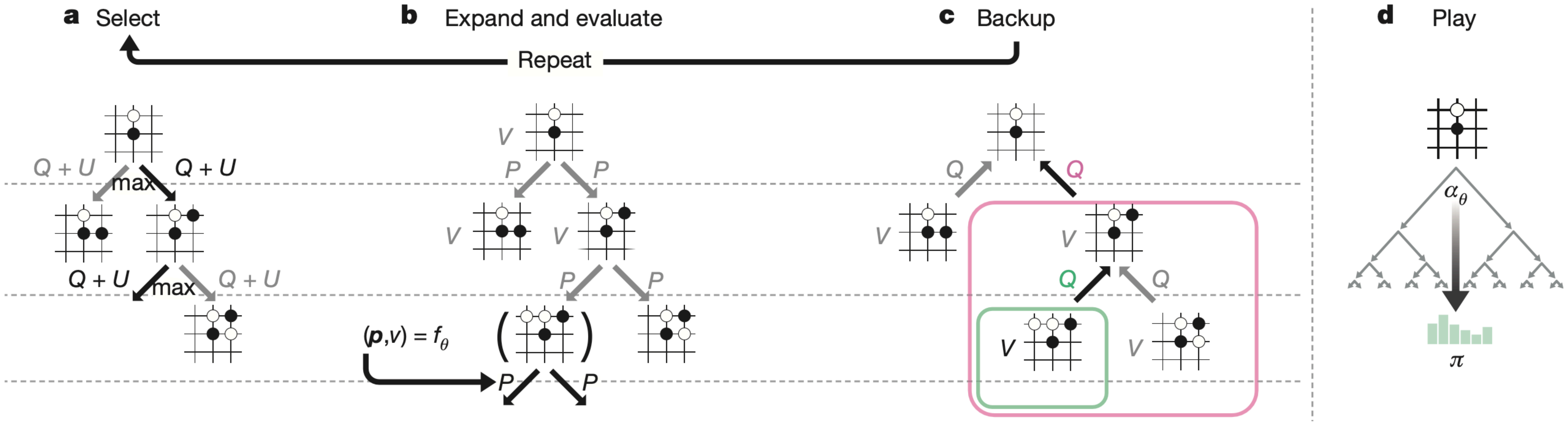
- Residual Neural Network f_θ
- **Input:** board representation s of the
  - current board position (black/white stones)
  - its history (necessary because of forbidden repetitions) → Go is not fully observable solely from the current position
- **Output:** move probabilities and value $(p, v) = f_\theta(s)$
  - p: vector of move probabilities, each probability of selecting the move $p_a = \Pr(a \mid s)$
  - v: scalar, probability of winning of the current player from position s



Credit: Silver et al. 2016

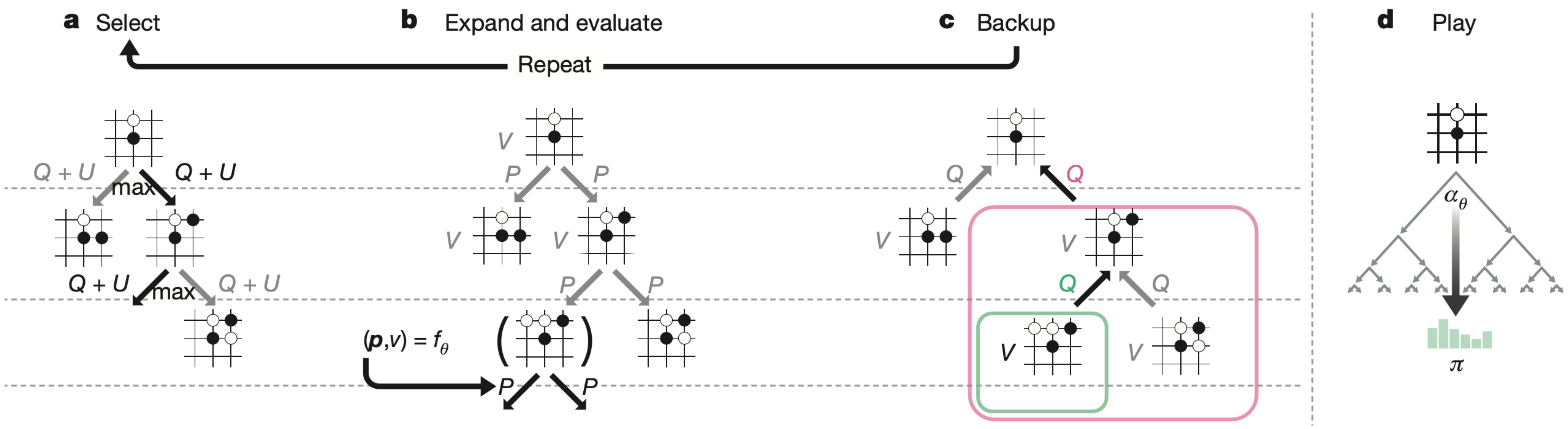
Monte Carlo Tree Search

- Objective: given a state s, output the probabilities π of playing each move
- Simulates games using the current NN f_θ
- For each edge (s, a) store:
  - Prior probability $P(s, a)$
  - Visit count $N(s, a)$
  - Action value $Q(s, a)$



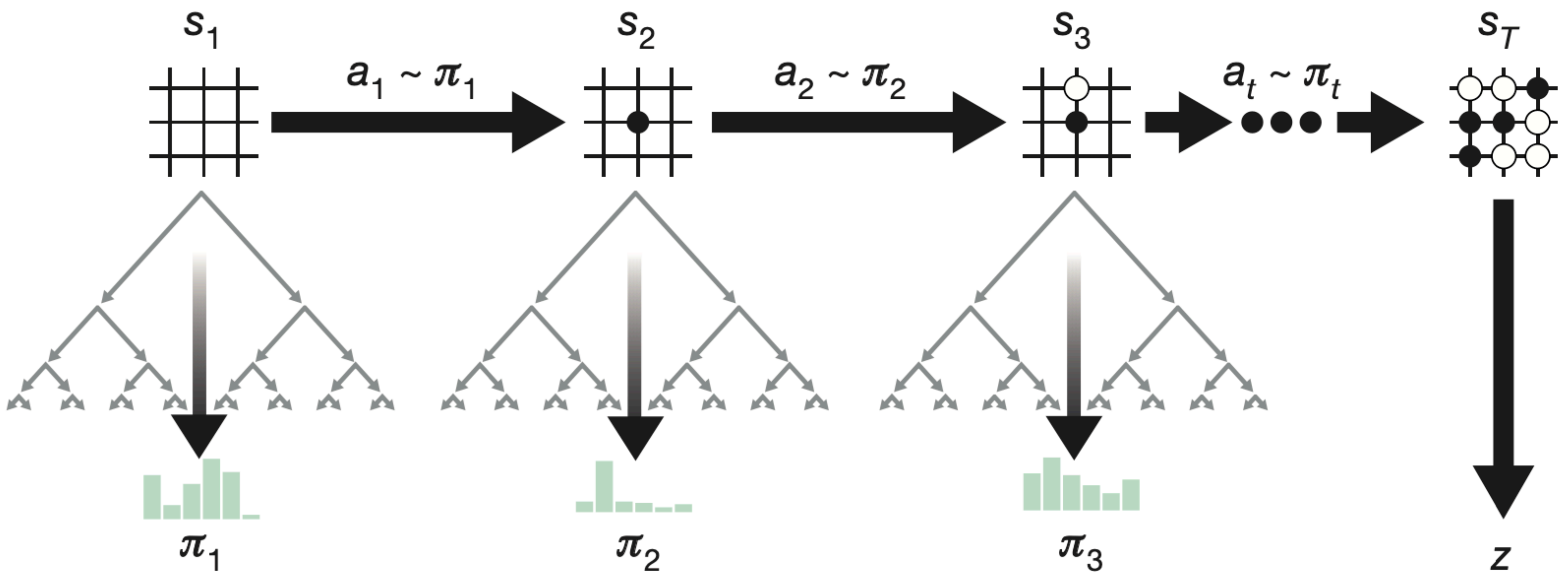
Monte Carlo Tree Search

- **Selection:**
  - Traverses the game tree by selecting moves that maximize an upper confidence bound $Q(s,a) + U(s,a)$
  - Where $U(s,a) \propto \dfrac{P(s,a)}{1 + N(s,a)}$
- **Expansion, Evaluation and Backup:**
  - Upon reaching a leaf, add all legal moves to the node; use the network to evaluate a random rotation or reflection of the state
  - Propagate the obtained value up the tree, updating Q-values and visit counts for each node along the path
- **Play:**
  - Once the search is complete, return the search probabilities
  - Sample move according to π



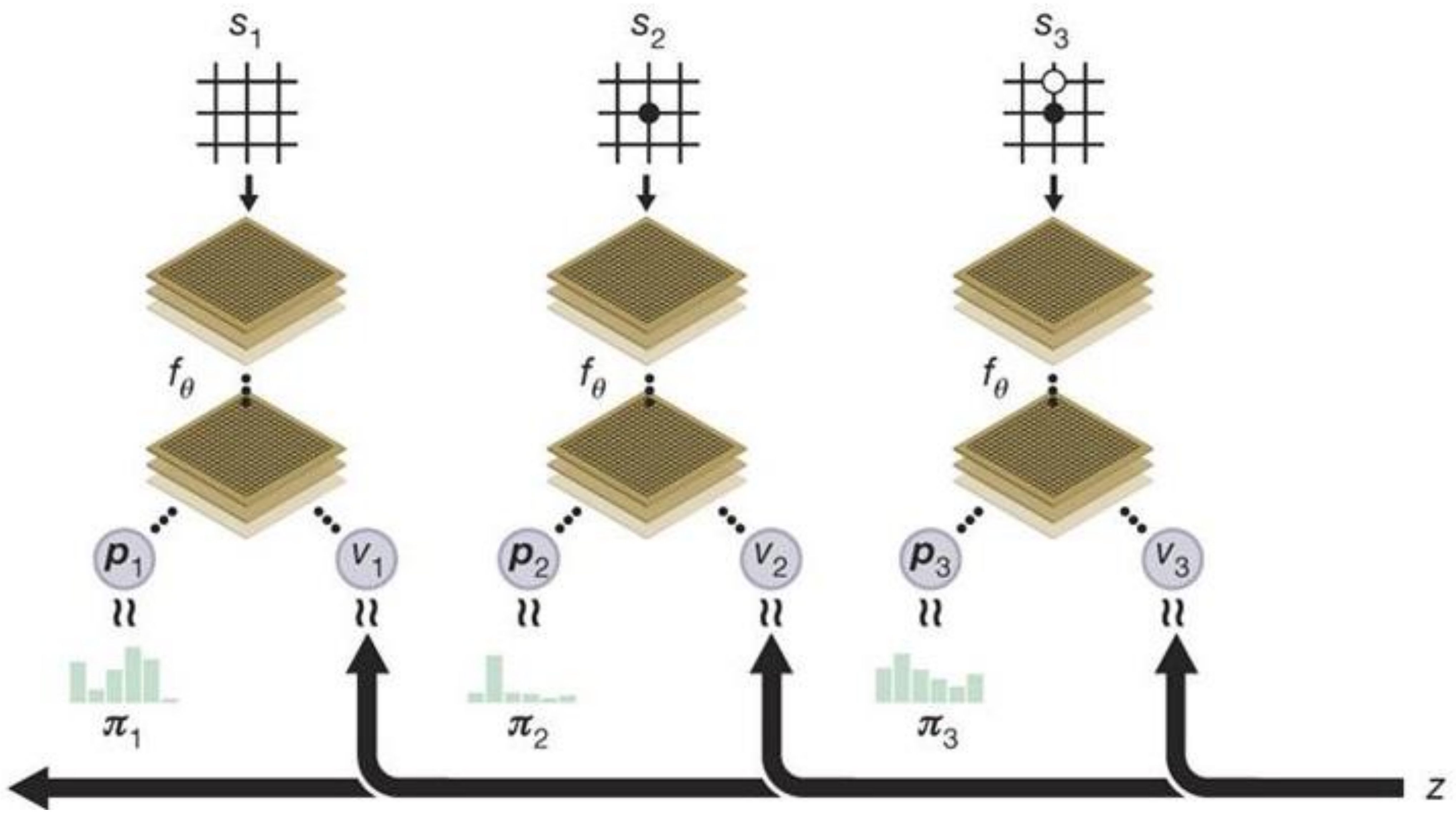
Self-play

- The agent plays full games against itself to generate training data
- At each turn:
    - predict move probabilities and state value for the current state s_t
    - use predicted probabilities to guide MCTS and obtain an improved policy π_t
    - select a move according to the search probabilities
- Terminal state is scored following the game's rules to compute the winner z



Self-play RL algorithm

- During self-play, store the data generated at each step t as a triple (s_t, π_t, z_t)
- Dataset is augmented to include rotations and reflections of each position
- Sample data uniformly among all time-steps of the last games
- Optimize to:
    - Minimize error between predicted value v_t and game winner z_t
    - Maximize similarity between network probabilities p_t and search probabilities π_t



Self-training Pipeline

AlphaGo Zero's self-play training pipeline consists of three main components, all executed asynchronously in parallel:

1. **Optimization:**

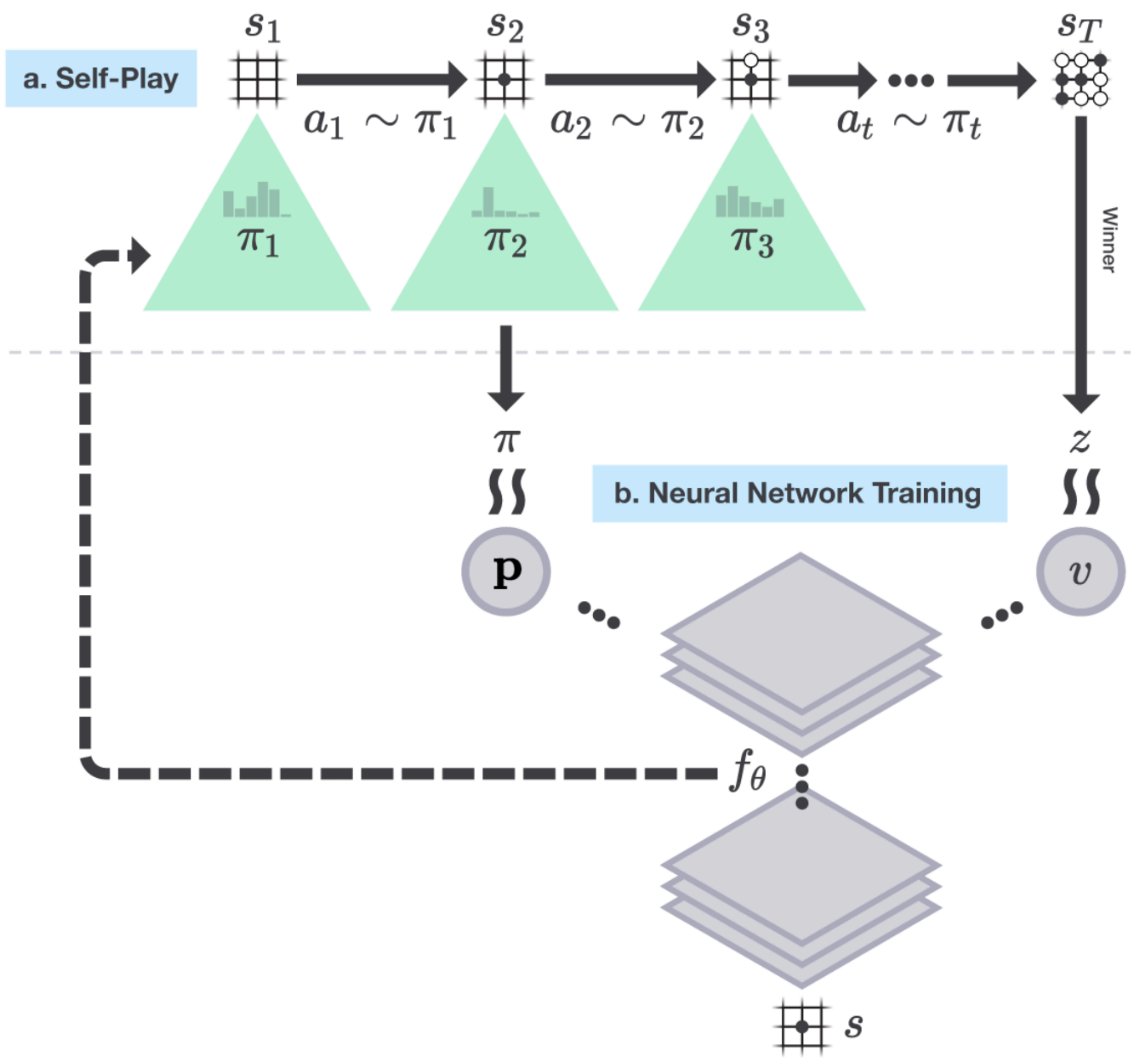
   Neural network parameters θ_i are continually optimized from recent self-play data

2. **Evaluation:**

   AlphaGo Zero players α_{θ_i} are continually evaluated

3. **Self-play:**

   The best performing player so far, α_{θ_*}, is used to generate new self-play data



Self-training Pipeline: Optimization

Training Worker

$$l = (z - v)^2 - \boldsymbol{\pi}^{\mathrm{T}} \log \boldsymbol{p} + c\|\theta\|^2$$

1. Optimization

- Train multiple networks in parallel using **stochastic gradient descent** with momentum and learning rate annealing
- Training data (s, π, z) sampled uniformly from the last 500,000 **self-play games**
- Produce a new **checkpoint** every 1000 training steps

Self-training Pipeline: Evaluation

Evaluator

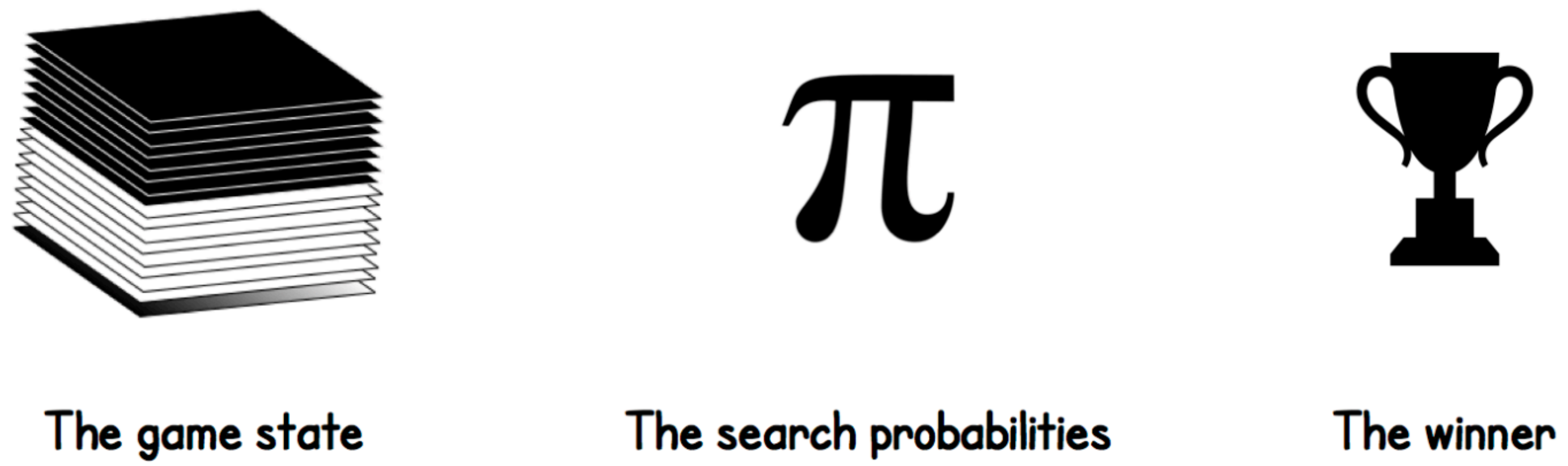
$$\pi' > \pi$$

2. Evaluation

- Each neural network checkpoint is evaluated against the current best f_{θ_*} in 400 games
- If the new player wins by a margin $>$ 55%, it becomes the new best player in the next self-play games

Self-training Pipeline: Self-play

Self-play Worker



s π z

3. Self-play

- The current best player α_{θ_*} determined by the evaluator is used to generate data
- In each iteration, α_{θ_*} plays 25,000 games of self-play using 1,600 simulations of MCTS to select each move

Image credit: Liam Hinzman, TDLS, https://www.youtube.com/watch?v=_x9bXso3wo4

Results: Overview

AlphaGo Zero model was tested and evaluated…

… first using:

- 20 block resNet
- 3 days of training
- A single machine in the Google Cloud with 4 TPUs

… then using (final model):

- 40 block resNet
- 40 days of training
- A single machine in the Google Cloud with 4 TPUs

… against:

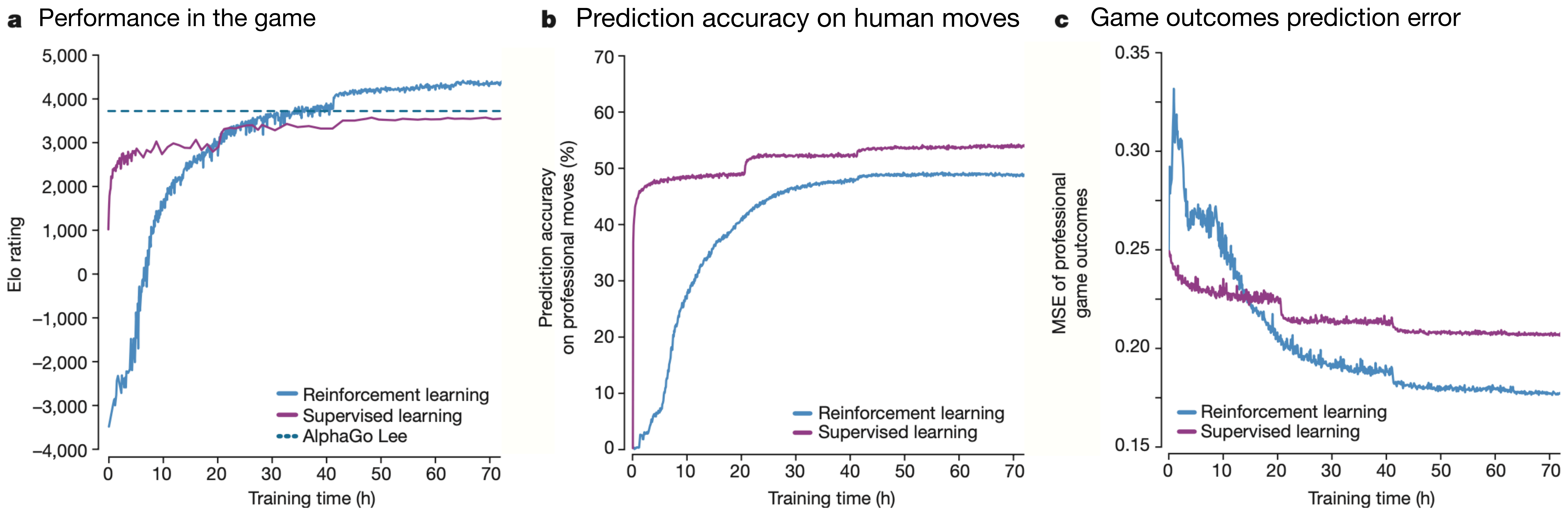
	Results vs Human Opponents	Description
AlphaGo Fan	5 - 0	Original AlphaGo implementation. Defeated Fan Hui.
AlphaGo Lee	4 - 1	Improved version of AlphaGo. Defeated Lee Sedol.
AlphaGo Master	60 - 0	Same architecture as AlphaGo Zero, trained on human data. Defeated strongest human professionals in online games.

and several previous Go programs like GnuGo, Pachi and Crazy Stone

Results: Empirical Evaluation

vs. AlphaGo Lee:

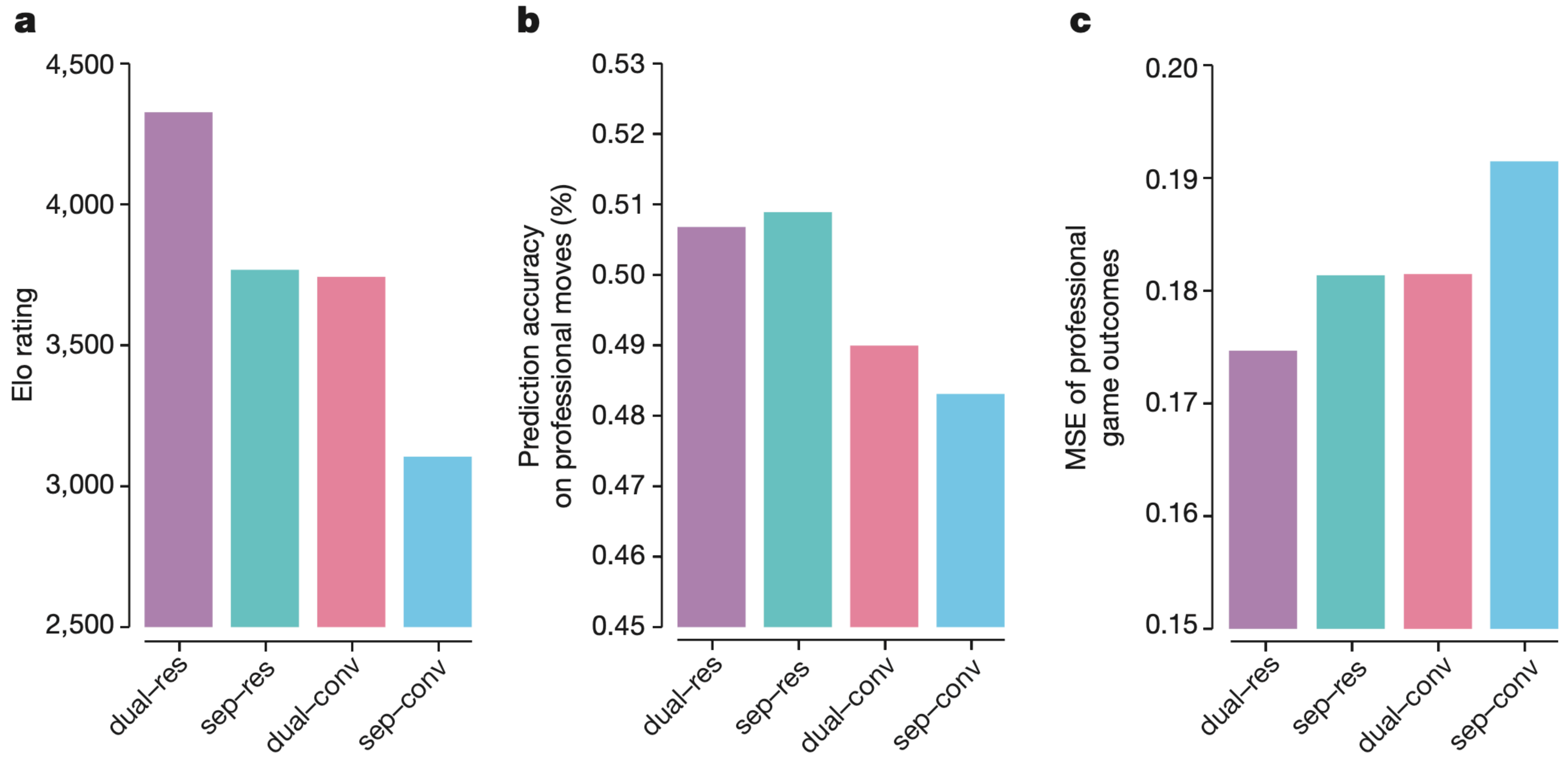
- Trained over several months
- Distributed over many machines and used 48 TPUs
- Defeated by 100 games to 0



Results: Architectures

- Dual = single NN for both policy and value
- Sep = separate policy and value networks

- Res = residual network
- Conv = convolutional network



Comparison of NN architectures in AlphaGo Zero and AlphaGo Lee

Results: Final performance

Final AlphaGo Zero model evaluated using an internal tournament:

vs. Raw network, AlphaGo Master, AlphaGo Lee, AlphaGo Fan, Crazy Stone, Pachi, GnuGo

Raw network:

- Player based solely on the raw neural network of AlphaGo Zero (without MCTS)
- It simply selects the move with maximum probability

AlphaGo Master:

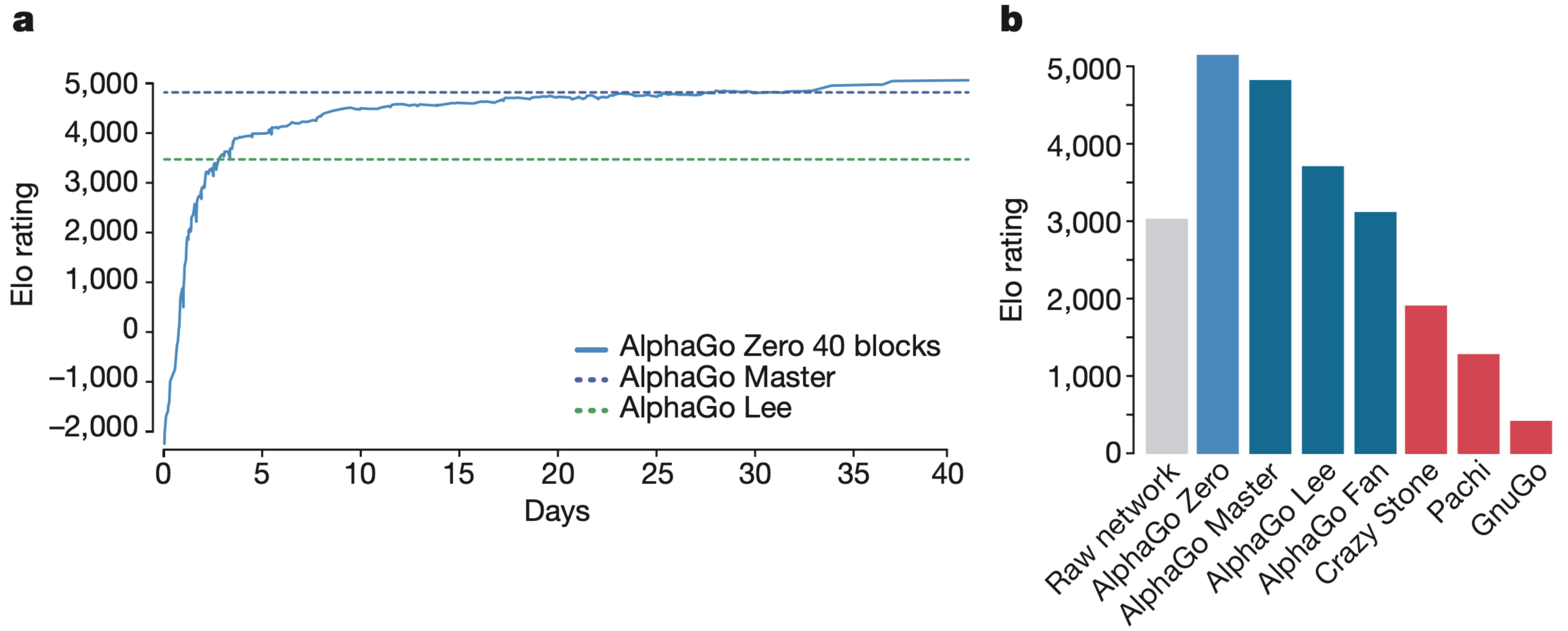
- Distributed on a single machine with 4 TPUs

AlphaGo Lee:

- Distributed over many machines and used 48 TPUs

AlphaGo Fan:

- Distributed over many machines and used 176 TPUs



Performance of AlphaGo Zero

Go knowledge learned by AlphaGo Zero

- The idea of AlphaGo is to discover what it means for a program to be able to learn for itself what knowledge is
- AlphaGo Zero discovered a good level of Go knowledge during its self-play training process:
  - Common patterns and opening of human Go knowledge
  - Non-standard strategies beyond traditional Go knowledge
- Learned a strategy qualitatively different from human play
- Developed **new pieces of knowledge** which were creative and novel in many ways, providing **new insights** into the game of Go

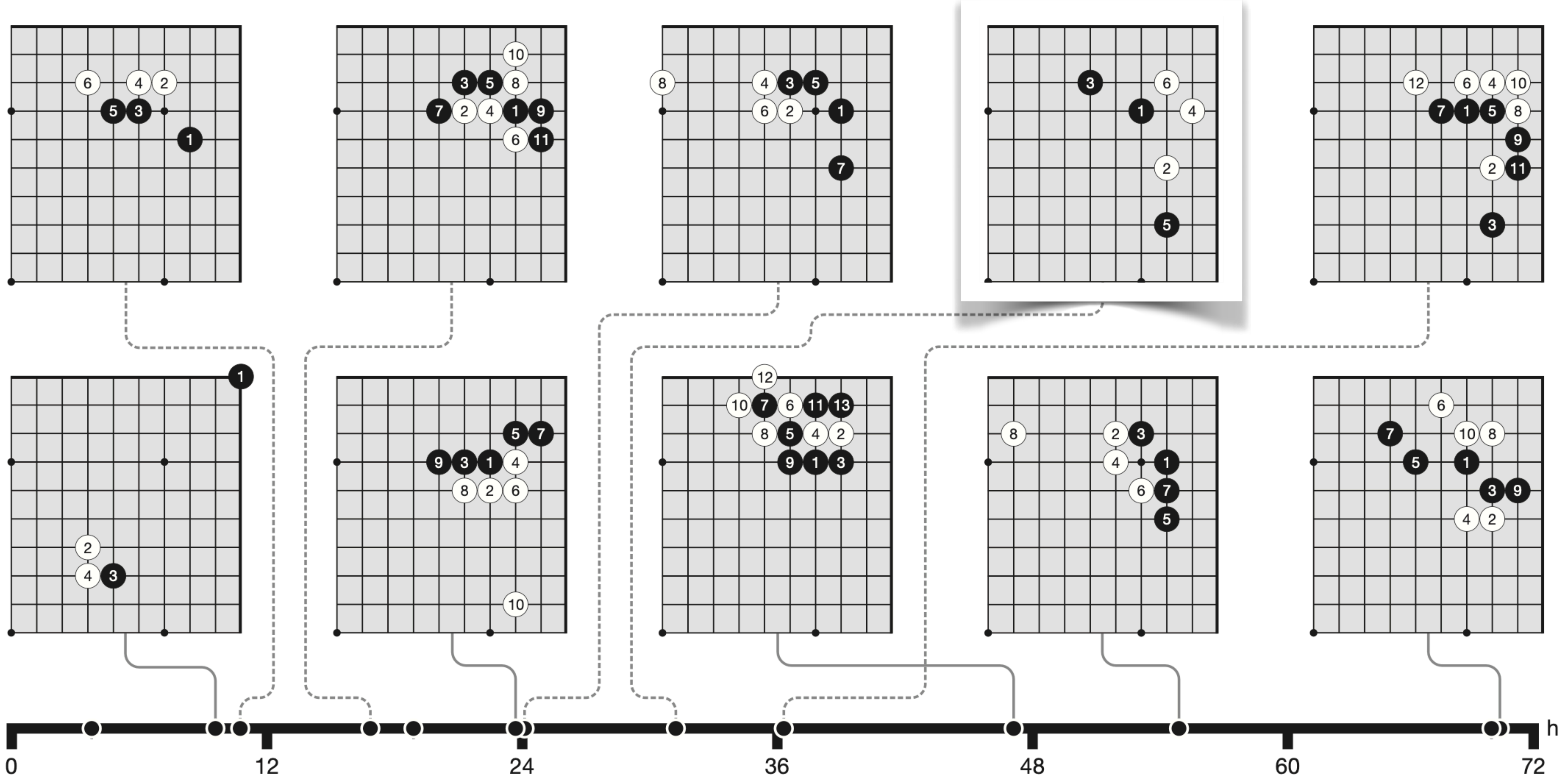
AlphaGo Zero: Discovering new knowledge

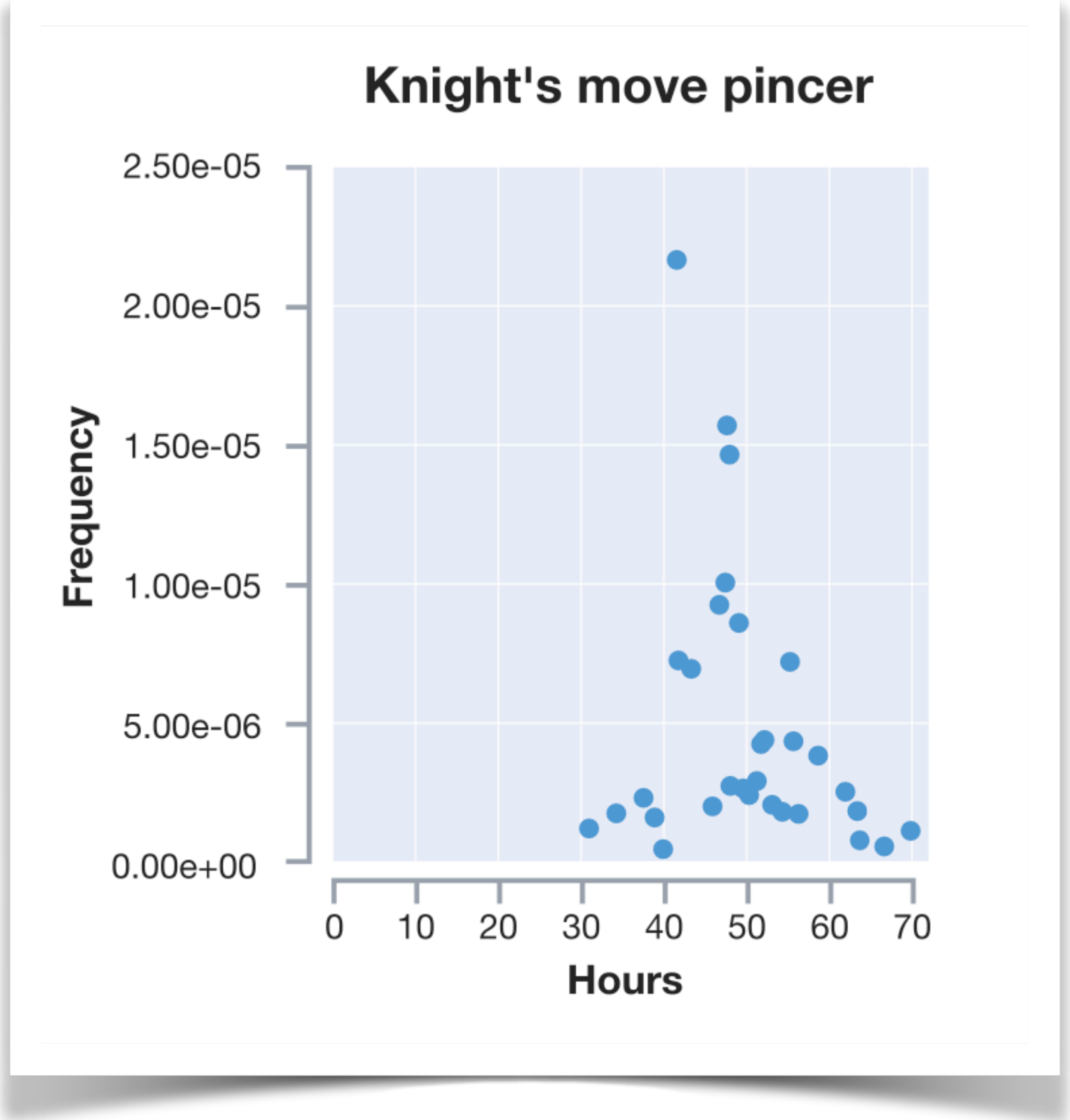
a.

Five human *joseki* discovered during training

b.

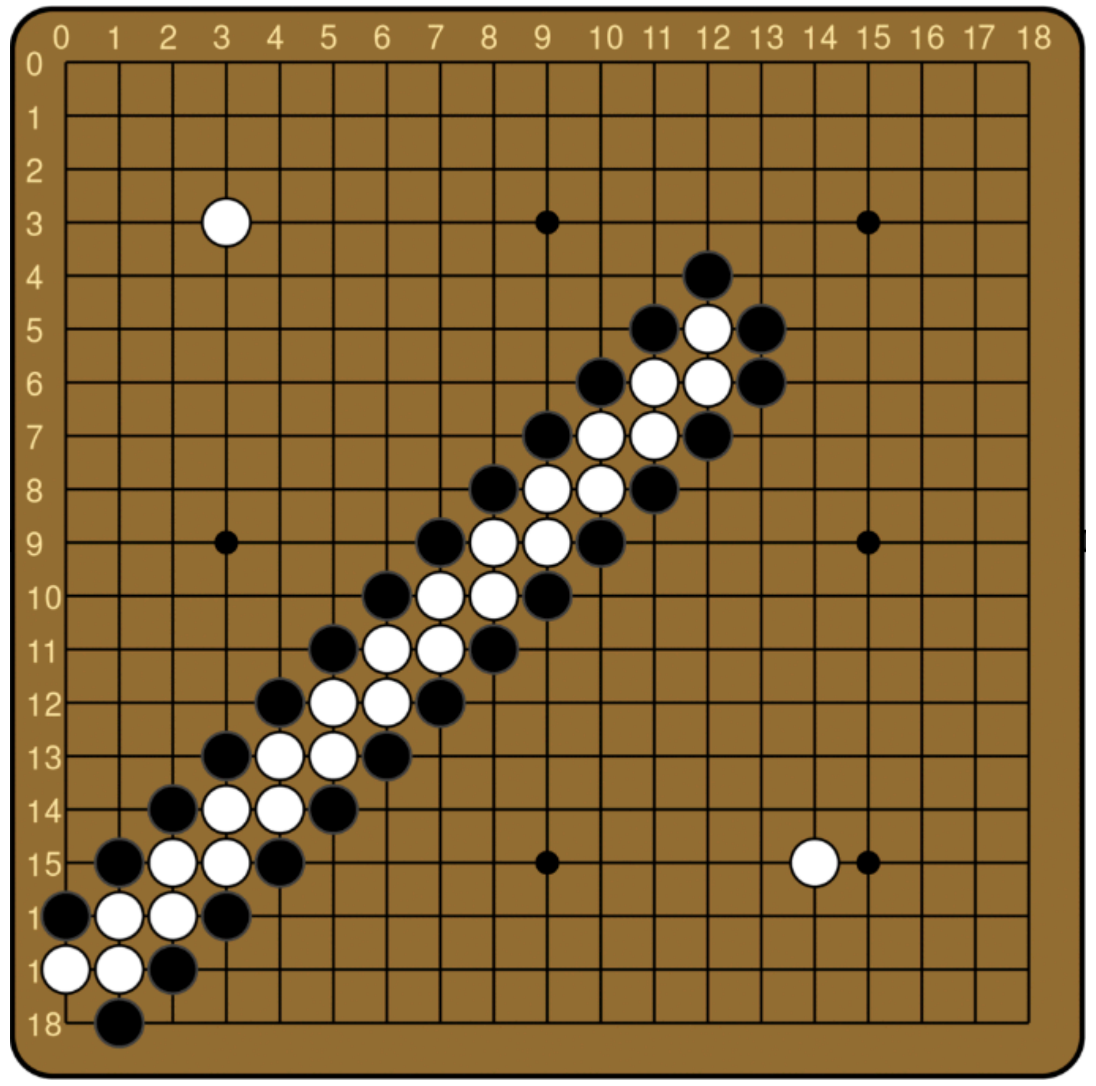
Five *joseki* favoured at different stages of self-play training

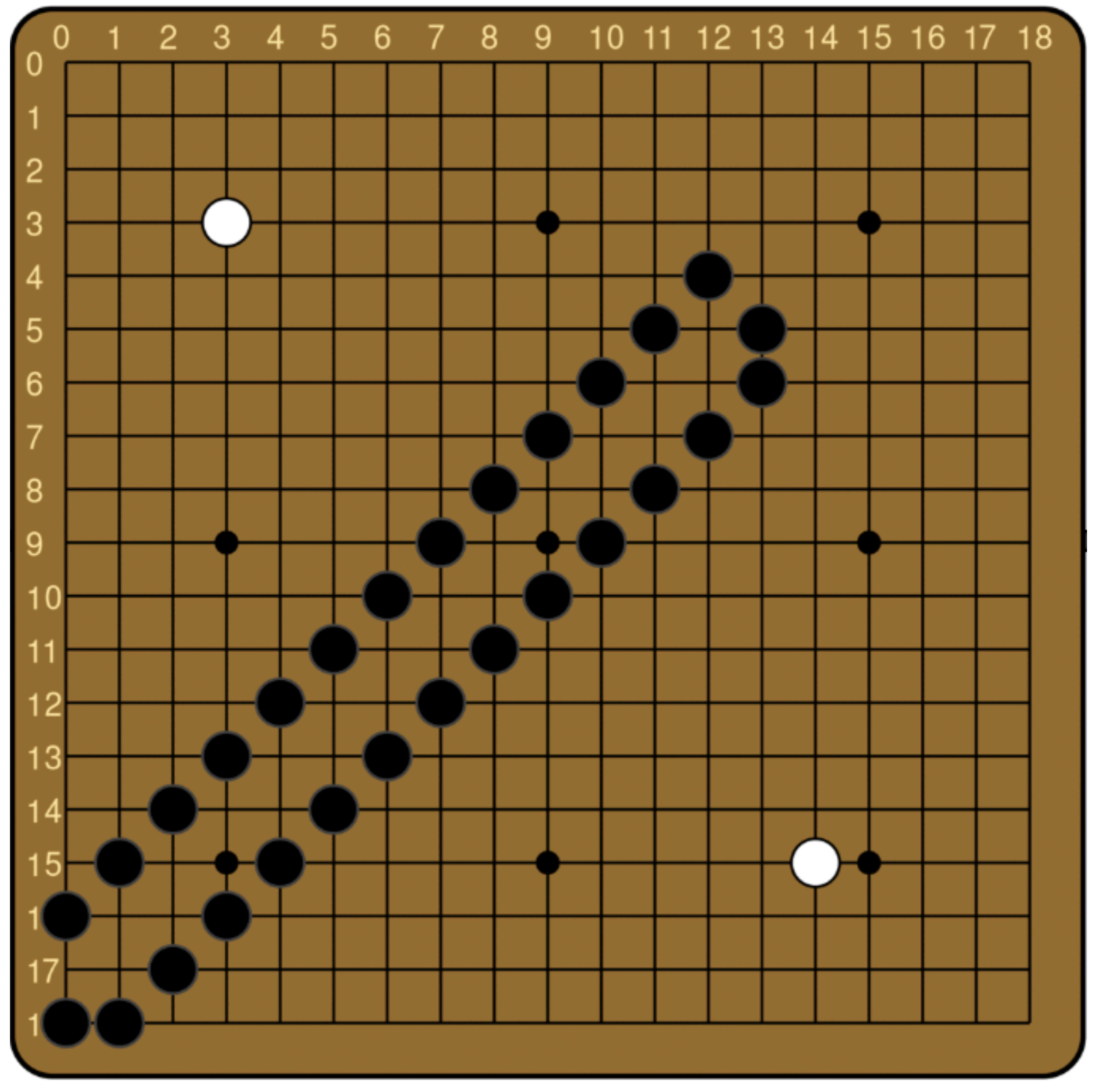




joseki = common corner sequences

AlphaGo Zero: Discovering new knowledge





shicho (japanese for "ladder") = capture sequence that may span the whole board

- One of the first elements of Go knowledge learned by humans
- Only understood by AlphaGo Zero much later in the training
- AlphaGo Zero discovered something that was obvious to humans much later, but it also discovered strategies that had never been obvious to human players at all

Advantages of AlphaGo Zero

- **Simple architecture:** single network for both policy and value, and simple MCTS algorithm
- **Efficiency:** no reliance on rollouts to evaluate states, saves computational resources during search
- **Self-improvement:** no dependency on human data, through self-play reinforcement, it learns
    - starting from random initial weights
    - using only the raw board state and history as input features
    - using the minimum amount of domain knowledge
- **Domain agnostic:** the employed method can be extended to other two-players games with little adjustments

Conclusions

- AlphaGo Zero highlights the potential of **reinforcement learning** in complex decision-making tasks
- It is possible to train to superhuman level, with **no human guidance** and given **no knowledge** of the domain beyond basic game rules
- AlphaGo Zero is able to **overcome** in performance both supervised learning systems and humans
  - Compared to training on human expert data, achieves ***better asymptotic performance*** requiring only a few more hours to train
  - Through self-play and starting tabula rasa, achieves ***superhuman performance*** in the span of a few days of training

    → with **less computational resources** than previous methods

    → rediscovering much of human Go knowledge as well as **novel strategies**

References

Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., ... & Hassabis, D. (2017). *Mastering the game of Go without human knowledge*. Nature, 550(7676), 354–359. https://doi.org/10.1038/nature24270

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press.

DeepMind. (2017, January 19). AlphaGo – The Movie | Full Documentary. YouTube. *https://www.youtube.com/watch?v=WXuK6gekU1Y*

Thanks for your attention